

Paralelización Fase 3 ProtSpaM

Alumna: Ana Izaguirre
Asesor: Jorge González Domínguez

Junio 2026

Índice

1. Contexto del proyecto	2
2. Objetivo	2
3. Diseño de la paralelización	2
3.1. Distribución del trabajo	2
4. Alternativas de distribución de datos	2
4.1. Opción A: lectura centralizada	2
4.2. Opción B: lectura distribuida	3
5. Representación de las estrategias	3
6. Metodología experimental	4
6.1. Entorno de ejecución	4
6.2. Configuración experimental y control de parámetros	4
7. Resultados preliminares	5
7.1. Resultados de la opción A	5
7.2. Resultados de la opción B	5
7.3. Comparación	7
8. Trabajo futuro	7
9. Conclusiones	8

1. Contexto del proyecto

En el avance anterior se realizó un análisis del flujo de ejecución de ProtSpaM con el objetivo de identificar las secciones con mayor coste computacional y estudiar posibles estrategias de paralelización.

A partir de dicho análisis se concluyó que las fases asociadas al cálculo de *spaced-words* y al cálculo de *matches* representan los principales candidatos para la aplicación de técnicas de computación paralela.

El presente avance se centra en la implementación y evaluación de la paralelización de la fase 3 mediante MPI, así como en el estudio de diferentes estrategias de distribución de datos entre procesos.

2. Objetivo

La fase 3 de ProtSpaM tiene como finalidad generar los *spaced-words* correspondientes a cada especie de entrada a partir de un conjunto común de patrones.

Dado que el procesamiento de cada especie puede realizarse de forma independiente, esta etapa presenta un elevado grado de paralelismo potencial y constituye un primer objetivo adecuado para la introducción de MPI dentro de la aplicación.

3. Diseño de la paralelización

3.1. Distribución del trabajo

La estrategia adoptada consiste en repartir el conjunto de especies entre los distintos procesos MPI disponibles.

Cada proceso calcula localmente los *spaced-words* correspondientes a las especies asignadas y, una vez finalizado el procesamiento, los resultados son reunidos para permitir la continuación del flujo de ejecución.

4. Alternativas de distribución de datos

4.1. Opción A: lectura centralizada

La estrategia de la Opción A se basa en un modelo de paralelización con punto de entrada único, en el cual el proceso **rank 0** asume la responsabilidad de la lectura completa de los datos de entrada.

En esta aproximación, el proceso principal carga todas las secuencias de las especies y construye la estructura de datos global necesaria para el cálculo de *spaced-words*. Posteriormente, este mismo proceso se encarga de distribuir subconjuntos de especies entre los distintos procesos MPI disponibles.

Cada proceso recibe un conjunto de especies asignado por **rank 0** y ejecuta de forma independiente el cálculo de *spaced-words* utilizando el mismo conjunto de patrones. Una vez finalizado el procesamiento local, los resultados son enviados

de vuelta al proceso `rank 0`, donde se recopilan para continuar con el flujo del programa.

4.2. Opción B: lectura distribuida

La estrategia de la Opción B adopta un enfoque descentralizado en la fase de entrada de datos. En este modelo, no existe un único proceso responsable de la lectura completa de las secuencias, sino que cada proceso MPI accede directamente a los datos que le han sido asignados.

Para ello, todos los procesos disponen de la lista global de archivos de entrada, pero cada `rank` realiza la lectura únicamente de las especies que le corresponden según el esquema de distribución definido. De esta forma, el coste de lectura se reparte entre todos los procesos del sistema.

Cada proceso calcula de manera independiente los *spaced-words* asociados a sus especies locales, utilizando el mismo conjunto de patrones que en la versión secuencial. Una vez finalizado el cálculo, los resultados se mantienen distribuidos o se recopilan según la fase posterior del algoritmo.

5. Representación de las estrategias

Opción A (Centralizada)

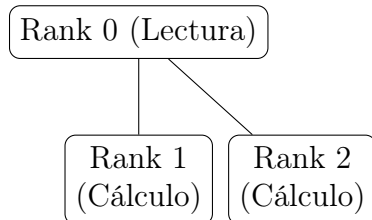


Figura 1: Modelo A: Centralizada.

Opción B (Distribuida)

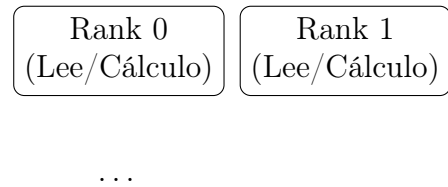


Figura 2: Modelo B: Distribuida.

6. Metodología experimental

6.1. Entorno de ejecución

Elemento	Descripción
Supercomputador	FinisTerra III (CESGA)
Organización	Centro de Supercomputación de Galicia (CESGA)
Ubicación	Santiago de Compostela, España
Sistema operativo	Linux (kernel 4.18.0-305.el8_4.x86_64)
Arquitectura	x86_64
CPU	Intel Xeon Platinum 8352Y @ 2.20 GHz
Núcleos totales	128 CPUs visibles en nodo de ejecución
Compilador	gcc 10.1.0 (Gentoo)
MPI	OpenMPI 5.0.9 (módulo del sistema)

Cuadro 1: Entorno de ejecución utilizado para la evaluación experimental.

6.2. Configuración experimental y control de parámetros

El conjunto de datos utilizado en este trabajo proviene del repositorio oficial de ProtSpaM, compuesto por secuencias biológicas en formato FASTA multifasta organizadas por especies.

La selección de entradas se realiza mediante el fichero `filelist`, definido según las indicaciones del *README* del proyecto. Este fichero no forma parte del dataset original, pero permite configurar de forma controlada los experimentos, utilizando subconjuntos de 10, 20 y 30 especies para evaluar la escalabilidad del sistema.

El algoritmo permite generar patrones aleatorios o cargarlos desde archivo; en este trabajo se utiliza de forma fija el fichero `patterns_clean.txt`, garantizando la reproducibilidad. Estos patrones determinan las posiciones relevantes para el cálculo de *spaced-words*.

El conjunto utilizado consta de 5 patrones de longitud 46, con 6 posiciones informativas (*weight*) y 40 posiciones no informativas (*don't-care positions*).

Los parámetros del algoritmo se mantienen en los valores por defecto de la implementación original:

- *Weight*: 6
- Number of don't-care positions: 40
- *Threshold*: 0
- Number of patterns: 5

Estos parámetros afectan directamente a la generación de *spaced-words* y al coste computacional de las fases de cálculo, especialmente en la cantidad de combinaciones generadas y en el filtrado de coincidencias.

Finalmente, las variables experimentales son el número de especies (controlado por `filelist`) y el número de procesos MPI, que determinan respectivamente el tamaño del problema y el grado de paralelización del sistema.

7. Resultados preliminares

7.1. Resultados de la opción A

En la tabla 2, se observa la ausencia de resultados para la especie 30 con (np) de 1, 2 y 4. Durante la ejecución de las pruebas de escalabilidad, estos casos presentaron errores de convergencia en la Fase 3, lo que impidió la obtención de métricas válidas de tiempo total. Este comportamiento sugiere que, para una mayor carga de datos (especie 30), el paralelismo introducido en configuraciones de pocos nodos no logra compensar el *overhead* de comunicación, resultando en inestabilidad. A partir de $np = 8$, el sistema logra estabilizarse, permitiendo realizar el análisis de *speedup* y eficiencia presentado.

Cuadro 2: Fase 3: Opción A de paralelización

Especies	np	Media fase 3	Media matches	Media total	Speedup	Eficiencia
10	1	115.856	75.1233	193.4912	1.000	1.0000
10	2	72.3049	75.1394	150.1164	1.6023	0.8012
10	4	52.0720	75.1794	130.0398	2.2249	0.5562
10	8	36.3753	74.8596	113.9766	3.1850	0.3981
10	16	26.2767	75.3703	104.3392	4.4091	0.2756
10	32	25.8904	75.1685	103.8244	4.4749	0.1398
20	1	309.1864	260.5516	576.0972	1.000	1.0000
20	2	205.5528	262.3690	475.0112	1.5042	0.7521
20	4	143.6552	264.2446	415.1706	2.1523	0.5381
20	8	83.5980	263.5314	354.3136	3.6985	0.4623
20	16	76.7915	265.2628	349.1558	4.0263	0.2516
20	32	77.0704	256.9934	341.0864	4.0117	0.1254
30	8	117.2136	576.1436	702.7504	2.8340	0.3540
30	16	91.5434	585.2320	686.5328	2.9010	0.1810
30	32	84.4331	585.3840	679.2670	2.9320	0.0910

7.2. Resultados de la opción B

En la tabla 3 se presentan los resultados obtenidos para la Opción B. Se observa una marcada fragilidad operativa frente a la Opción A, especialmente ante cargas de trabajo elevadas: para el caso de 30 especies, los experimentos con np de 1, 2, 4, 8 y 16 no lograron completar la ejecución debido a errores en la Fase 3. Esta

inestabilidad, que se manifiesta de forma recurrente en configuraciones de baja paralelización, revela que el esquema de lectura distribuida es significativamente más sensible a las limitaciones de memoria del sistema que la lectura centralizada. Como consecuencia, la Opción B resulta menos versátil y presenta un umbral operativo superior, requiriendo un mayor número de procesadores ($np = 32$) para garantizar la estabilidad del cálculo frente a volúmenes de datos exigentes.

Cuadro 3: Fase 3: Opción B de paralelización

Especies	np	Media fase 3	Media matches	Media total	SpeedUp	Eficiencia
10	1	115.5980	75.6583	193.6618	1.0000	1.0000
10	2	73.8093	76.0177	151.2390	1.5662	0.7831
10	4	54.8359	75.7346	131.3250	2.1081	0.5270
10	8	39.2691	76.1412	115.9560	2.9437	0.3680
10	16	28.5353	77.2979	106.1778	4.0511	0.2532
10	32	28.4057	77.1735	106.0628	4.0695	0.1272
20	1	314.2664	268.8988	589.3794	1.0000	1.0000
20	2	213.7504	271.1074	487.3304	1.4702	0.7351
20	4	144.2288	270.2190	415.8880	2.1789	0.5447
20	8	89.2194	271.8170	361.8026	3.5224	0.4403
20	16	79.5801	271.2628	351.4444	3.9491	0.2468
20	32	79.7755	271.6070	351.7484	3.9394	0.1231
30	32	92.5382	605.1006	698.0124	2.8534	0.0892

7.3. Comparación

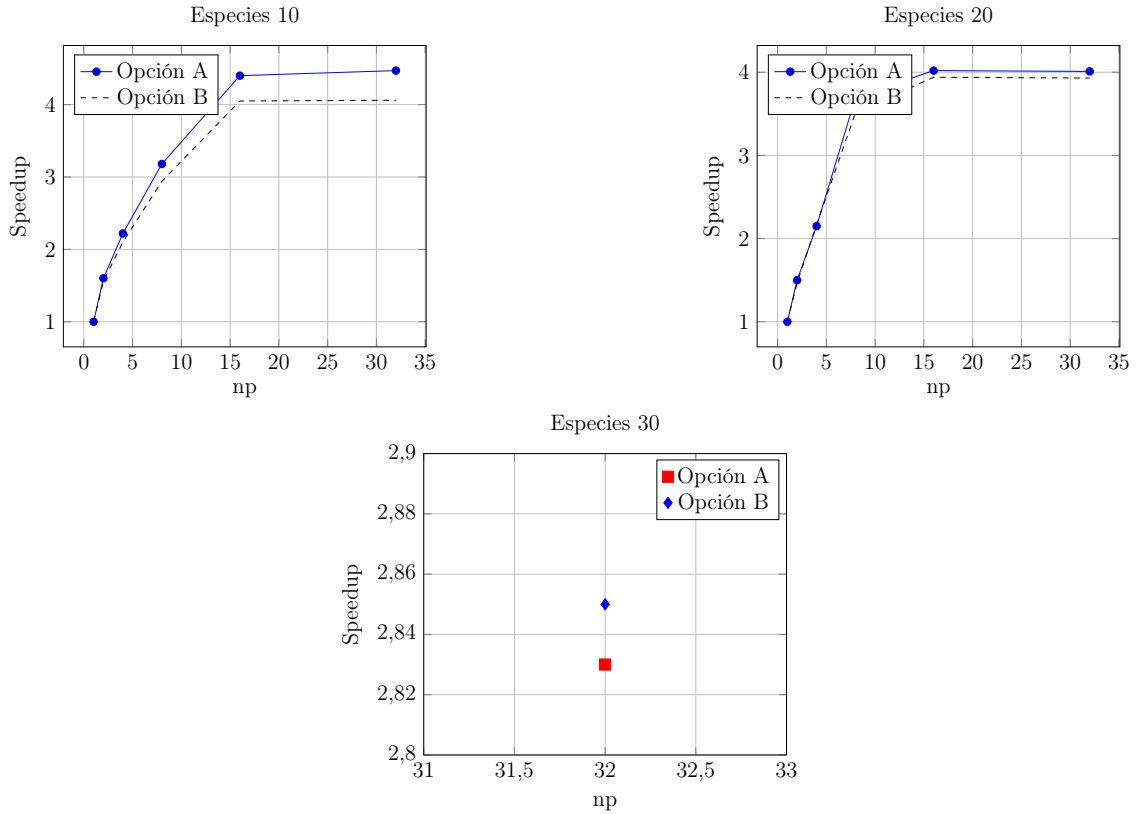


Figura 3: Comparativa de Speedup por número de especies y opción de paralelización.

Discusión de los resultados

Para volúmenes de 10 y 20 especies, ambas estrategias mantienen una escalabilidad estable, aunque con una degradación de la eficiencia a 32 nodos debido al *overhead* de comunicación. Ante una carga de 30 especies, los errores en configuraciones de pocos nodos evidencian una saturación en la gestión de memoria de la Fase 3. En este escenario, la **Opción A** resulta superior: no solo optimiza ligeramente los tiempos totales en el punto de convergencia ($np = 32$), sino que demuestra una mayor robustez operativa al mantenerse estable en 8 y 16 nodos, donde la Opción B resulta inviable.

8. Trabajo futuro

Tras el análisis de escalabilidad de la Fase 3, la siguiente etapa del proyecto se centrará en la paralelización de la Fase 4. Esta fase abordará el cálculo de *matches* entre especies y la generación de la matriz de distancias final. El objetivo principal será mitigar el *overhead* de comunicación identificado en las pruebas anteriores, optimizando la gestión de memoria para garantizar que el rendimiento del algorit-

mo sea estable ante volúmenes de datos superiores, evitando así las inestabilidades observadas en configuraciones de alta carga.

9. Conclusiones

Se ha implementado una versión paralela de la Fase 3 de ProtSpaM mediante MPI, evaluando dos estrategias de distribución de datos (Opción A y B). El análisis de los resultados muestra que ambas configuraciones presentan una escalabilidad similar, con una degradación de la eficiencia esperada conforme aumenta el número de procesadores.

Ante las pruebas realizadas, se determina que la **Opción A** es la estrategia más adecuada para continuar con el desarrollo del proyecto. Esta opción demostró una mayor capacidad de escalado, logrando completar la ejecución en configuraciones donde la Opción B presentaba errores, y ofreciendo, en términos generales, tiempos de ejecución totales menores(con $np = 32$) . Estos resultados la consolidan como la base más robusta para la futura paralelización de la Fase 4.